

Experiment 1:

8-Puzzle Problem

Aim:

To write a python program to solve the 8-puzzle problem.

Objective:

To understand heuristic-based search techniques and apply the A* algorithm with the Manhattan distance heuristic to solve the 8-puzzle sliding tile problem.

Algorithm:

Step1: Dequeue the state with the lowest priority from the frontier.

Step2: If the dequeued state is the goal state, return the solution path.

Step3: Add the dequeued state to the explored set.

Step4: Generate all possible successor states by swapping the empty tile with its neighboring tiles.

Step5: For each successor state, calculate its priority based on the Manhattan distance and add it to the frontier if it has not been explored before.

Program:

```
from queue import PriorityQueue

goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

def heuristic(state):
    h = 0
    for i in range(9):
        if state[i] != 0:
            x1, y1 = divmod(i, 3)
            x2, y2 = divmod(state[i]-1, 3)
            h += abs(x1-x2) + abs(y1-y2)
    return h

def successors(state):
    moves = []
    i = state.index(0)
    r, c = divmod(i, 3)

    directions = [(-1,0),(1,0),(0,-1),(0,1)]
```

```

for dr, dc in directions:
    nr, nc = r+dr, c+dc
    if 0 <= nr < 3 and 0 <= nc < 3:
        ni = nr*3 + nc
        new = list(state)
        new[i], new[ni] = new[ni], new[i]
        moves.append(tuple(new))

return moves

def solve(start):
    pq = PriorityQueue()
    pq.put((heuristic(start), 0, start, [start]))
    visited = set()

    while not pq.empty():
        f, g, state, path = pq.get()

        if state == goal:
            return path

        if state in visited:
            continue

        visited.add(state)

        for nxt in successors(state):
            if nxt not in visited:
                pq.put((g+1+heuristic(nxt), g+1, nxt, path+[nxt]))

    return None

start = (1, 2, 3,
        5, 6, 0,
        7, 8, 4)

solution = solve(start)

if solution:
    print("Solution Path:\n")
    for state in solution:
        for i in range(0, 9, 3):
            print(state[i], state[i+1], state[i+2])
        print()
else:
    print("No Solution")

```

Output:

1 2 3

5 6 0

7 8 4

1 2 3

5 0 6

7 8 4

1 2 3

0 5 6

7 8 4

1 2 3

7 5 6

0 8 4

1 2 3

7 5 6

8 0 4

1 2 3

7 5 6

8 4 0

1 2 3

7 5 0

8 4 6

Result: Hence, the simple python program for 8- puzzle is done

Exp: 02

Write the Python Program to solve 8-Queen Problem

Aim:

To write a python program to solve 8-Queen problem.

Objective:

To understand the backtracking search technique by placing 8 queens on an 8x8 chessboard such that no two queens attack each other.

Algorithm:

Step1: Start with an empty board of size 8x8.

Step2: Place the first queen in the first row and the first column.

Step3: For each subsequent row, check all possible columns to place the queen, and backtrack if a valid position cannot be found.

Step4: A valid position is a column where no other queens are in the same row, column, or diagonal.

Step5: If no solution is found, return failure

Program:

N = 8

```
board = [[0] * N for _ in range(N)]
```

```
def is_safe(row, col):
```

```
    for i in range(col):
```

```
        if board[row][i]:
```

```
            return False
```

```
    i, j = row, col
```

```
    while i >= 0 and j >= 0:
```

```
        if board[i][j]:
```

```
            return False
```

```
    i -= 1
```

```
    j -= 1
```

```

i, j = row, col
while i < N and j >= 0:
    if board[i][j]:
        return False
    i += 1
    j -= 1

return True

def solve(col):
    if col >= N:
        return True

    for row in range(N):
        if is_safe(row, col):
            board[row][col] = 1

            if solve(col + 1):
                return True

            board[row][col] = 0

    return False

if solve(0):
    print("Solution:")
    for row in board:
        print(*row)
else:
    print("No solution exists")

```

Output:

```
1 0 0 0 0 0 0 0
0 0 0 0 0 0 1 0
0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 1
0 1 0 0 0 0 0 0
0 0 0 1 0 0 0 0
0 0 0 0 0 1 0 0
0 0 1 0 0 0 0 0
```

Result: Hence, the simple python program for 8- Queen problem is done

Exp:03

Write the Python Program for Water Jug Problem

Aim:

To write a python program for water jug problem

Objective:

To understand state-space search by finding a sequence of operations (fill, empty, pour) that measures a target quantity of water using two jugs of different capacities.

Algorithm:

Step1: Start with two empty jugs of different capacities, and a target quantity of water to measure.

Step2: Fill one jug to its maximum capacity.

Step3: Pour the water from the full jug into the other jug until it is full or the first jug is empty.

Step4: If the second jug is full, pour out the water to empty it.

Step5: Repeat steps 2-4, filling and pouring water between the two jugs, until the desired quantity of water is measured or all possible combinations are tried.

Program:

```
from collections import deque

def water_jug():
    visited = set()
    queue = deque()

    start = (0, 0)
    goal = 2

    queue.append((start, [start]))
    visited.add(start)

    while queue:
        (a, b), path = queue.popleft()
```

```

if a == goal or b == goal:
    return path

next_states = [
    (4, b),          # Fill 4L jug
    (a, 3),          # Fill 3L jug
    (0, b),          # Empty 4L jug
    (a, 0),          # Empty 3L jug
    (max(0, a-(3-b)), min(3, b+a)), # Pour 4L -> 3L
    (min(4, a+b), max(0, b-(4-a))) # Pour 3L -> 4L
]

for state in next_states:
    if state not in visited:
        visited.add(state)
        queue.append((state, path + [state]))

path = water_jug()

print("Solution:")
for state in path:
    print(state)    if col >= N:
    return True

for row in range(N):
    if is_safe(row, col):
        board[row][col] = 1

    if solve(col + 1):
        return True

    board[row][col] = 0

```

```
    return False

if solve(0):
    print("Solution:")
    for row in board:
        print(*row)
else:
    print("No solution exists")
```

Output:

```
(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
```

Result:Hence, the simple python program for water jug problem is done

Exp: 04

Write the Python Program for Crypt-Arithmetic Problem.

Aim:

To write a python program for Crypt-Arithmetic problem.

Objective:

To understand constraint satisfaction problems (CSP) by assigning unique digits to letters such that the resulting arithmetic equation holds true.

Algorithm:

Step 1: Write down the arithmetic problem in full, including any carry-overs.

Step 2: Replace each letter with a unique digit from 0 to 9.

Step 3: Generate all possible combinations of digits for each letter, subject to any constraints from the carry-overs and the rules of arithmetic.

Step 4: For each combination of digits, evaluate the arithmetic problem and check if it satisfies the given constraints.

Step 5: If a solution is found, output the values of the letters and the arithmetic problem, and exit the algorithm.

Step 6: If no solution is found, output a message indicating that no solution exists.

Program:

```
from itertools import permutations
```

```
letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')
```

```
digits = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
```

```
for p in permutations(digits, 8):
```

```
    S, E, N, D, M, O, R, Y = p
```

```
    if S == 0 or M == 0:
```

```
        continue
```

```
    SEND = 1000*S + 100*E + 10*N + D
```

$$\text{MORE} = 1000 * M + 100 * O + 10 * R + E$$

$$\text{MONEY} = 10000 * M + 1000 * O + 100 * N + 10 * E + Y$$

```

if SEND + MORE == MONEY:
    print("SEND =", SEND)
    print("MORE =", MORE)
    print("MONEY =", MONEY)
    break

```

Output:

SEND = 9567

MORE = 1085

MONEY = 10652

After defining the solve_cryptarithmic function, the code provides an example usage using a

cryptarithmic puzzle: "SEND + MORE = MONEY." It calls the solve_cryptarithmic

function with the puzzle string and stores the result in the mapping variable.

If a solution is found (i.e., mapping is not None), it prints the mapping dictionary and the puzzle

string with the letters replaced by their corresponding digits using the translate method.

If no solution is found, it prints "No solution found."

Result:

The output has verified.

Exp :05

Write the python program for Missionaries Cannibal problem.

Aim:

To write a python program for Missionaries Cannibal problem.

Objective:

To understand state-space search with constraints by transporting 3 missionaries and 3 cannibals across a river using a boat, without cannibals ever outnumbering missionaries on either bank.

Algorithm:

Step 1: Create a data structure to represent the state of the problem, including the number of missionaries and cannibals on each side of the river and the position of the boat.

Step 2: Create a data structure to represent the state space of the problem, including the initial state, the goal state, and the possible transitions between states.

Step 3: Use a search algorithm, such as depth-first search or breadth-first search, to explore the state space and find a solution.

Step 4: At each step of the search, generate all possible successor states from the current state by applying one of the allowed boat movements (two missionaries, two cannibals, or one of each).

Step 5: Check if the successor state is valid (i.e., no more cannibals than missionaries on either side of the river).

Step 6: If the successor state is valid, add it to the search tree and continue the search.

Step 7: If the successor state is the goal state, return the path from the initial state to the goal state.

Step 8: If there are no more possible successor states to explore, backtrack to the previous state and try another possible movement.

Program:

```
left_m = 3
```

```
left_c = 3
```

```
right_m = 0
```

```
right_c = 0
boat = "left"
```

```
print("Game Start\n")
print("Now the task is to move all of them to right side of the river")
print("Rules:")
print("1. The boat can carry at most two people")
print("2. If cannibals are greater than missionaries, the missionaries will be eaten")
print("3. The boat cannot cross the river by itself with no people on board\n")
```

```
while True:
```

```
    print("\nLeft Bank : ", "M " * left_m + "C " * left_c)
    print("Right Bank:", "M " * right_m + "C " * right_c)
```

```
    if left_m == 0 and left_c == 0:
        print("\nCongratulations! You solved the problem.")
        break
```

```
    print("\n", boat.capitalize(), "side ->", "Right" if boat == "left" else "Left", "side
river travel")
```

```
    m = int(input("Enter number of Missionaries travel => "))
    c = int(input("Enter number of Cannibals travel => "))
```

```
    if m + c == 0 or m + c > 2:
        print("Invalid input please retry !!")
        continue
```

```
    if boat == "left":
        if m > left_m or c > left_c:
            print("Invalid input please retry !!")
            continue
```

```

left_m -= m
left_c -= c
right_m += m
right_c += c
boat = "right"

else:
    if m > right_m or c > right_c:
        print("Invalid input please retry !!")
        continue

    right_m -= m
    right_c -= c
    left_m += m
    left_c += c
    boat = "left"

if (left_m > 0 and left_c > left_m) or (right_m > 0 and right_c > right_m):
    print("\nCannibals ate the Missionaries!")
    print("Game Over.")
    break

```

Output:

Game Start

Now the task is to move all of them to right side of the river

Rules:

1. The boat can carry at most two people
2. If cannibals are greater than missionaries, the missionaries will be eaten
3. The boat cannot cross the river by itself with no people on board

Left Bank : M M M C C C

Right Bank:

Left side -> Right side river travel

Enter number of Missionaries travel =>

Result:

The program was executed and the output was found to be correct

Exp: 06

Write the python program for Vacuum Cleaner Problem.

Aim:

To write a python program for vacuum cleaner problem.

Objective:

To understand the concept of a simple reflex agent that perceives its environment (room status) and acts (suck/move) based on condition-action rules.

Algorithm:

Step 1: Create a data structure to represent the state of the problem, including the current position of the vacuum cleaner and the state of each square in the room (clean or dirty).

Step 2: Create a data structure to represent the state space of the problem, including the initial state, the goal state, and the possible transitions between states.

Step 3: Use a search algorithm, such as depth-first search or breadth-first search, to explore the state space and find a solution.

Step 4: At each step of the search, generate all possible successor states from the current state by applying one of the allowed actions (move up, down, left, right, or clean).

Step 5: Check if the successor state is valid (i.e., the vacuum cleaner cannot move outside the room, and it can only clean dirty squares).

Step 6: If the successor state is valid, add it to the search tree and continue the search.

Step 7: If the successor state is the goal state, return the path from the initial state to the goal state.

Step 8: If there are no more possible successor states to explore, backtrack to the previous state and try another possible action.

Program:

```
def vacuum_cleaner():  
    rooms = {'A': 'Dirty', 'B': 'Dirty'}  
    current = 'A'
```

```

while True:
    print("\nCurrent Location:", current)
    print("Room Status:", rooms)

    if rooms[current] == 'Dirty':
        print("Action: Clean")
        rooms[current] = 'Clean'
    else:
        print("Action: Move")

    if current == 'A':
        current = 'B'
    else:
        current = 'A'

    if rooms['A'] == 'Clean' and rooms['B'] == 'Clean':
        print("\nBoth rooms are clean.")
        break

```

vacuum_cleaner()

Output:

Current Location: A
Room Status: {'A': 'Dirty', 'B': 'Dirty'}
Action: Clean

Current Location: A
Room Status: {'A': 'Clean', 'B': 'Dirty'}
Action: Move

Current Location: B
Room Status: {'A': 'Clean', 'B': 'Dirty'}

Action: Clean

Both rooms are clean.

Result:

Hence, the simple python program for Vacuum Cleaner Problem was done

Exp: 07

Write the Python Program to implement BFS

Aim:

To write a python program to implement BFS.

Objective:

To understand level-order traversal of a graph using a queue-based (FIFO) approach

Algorithm:

Step 1: Initialize a queue data structure and a set to keep track of visited nodes.

Step 2: Add the starting node to the queue and to the set of visited nodes.

Step 3: While the queue is not empty, dequeue the next node from the queue.

Step 4: For each neighbor of the dequeued node, if it has not been visited yet, add it to the queue and mark it as visited.

Step 5: Repeat steps 3-4 until the queue is empty

Program:

```
from collections import deque
```

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': ['F'],  
    'F': []  
}
```

```
def bfs(graph, start):  
    visited = set()  
    queue = deque([start])  
  
    visited.add(start)
```

```
while queue:
    node = queue.popleft()
    print(node, end=" ")

    for neighbor in graph[node]:
        if neighbor not in visited:
            visited.add(neighbor)
            queue.append(neighbor)
```

```
bfs(graph, 'A')
```

Output:

A B C D E F

Result:

Thus, the was executed and found the output was found to be correct.